MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 19 — Expressions Régulières

Le langage dans le langage — chercher, valider, extraire

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

17 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 19 — Les Expressions Régulières **2**
0.1 Les métacaractères fondamentaux 2
0.2 Les fonctions du module re 2
0.3 re.findall() et re.finditer() 3
0.4 Les groupes de capture 4
0.5 re.sub() — Remplacer 5
0.6 re.split() — Découper 6
0.7 Compiler un pattern — re.compile() 7
0.8 Patterns pratiques — Validations courantes 8
0.9 Ce qui m'a le plus surpris dans ce chapitre 9
0.10 Résumé du Chapitre 19 9

Chapitre 19 — Les Expressions Régulières

À retenir

Une **expression régulière** (regex) est un mini-langage permettant de décrire des **motifs** dans du texte. Valider un email, extraire des numéros de téléphone, nettoyer des données, parser des logs — les regex font en une ligne ce qui nécessiterait des dizaines de conditions. C'est l'un des outils les plus puissants du traitement de texte, présent dans presque tous les langages de programmation.

Les métacaractères fondamentaux

`.` N'importe quel caractère (sauf \n)	`^` Début de chaîne	`$` Fin de chaîne
`*` 0 ou plusieurs fois	`+` 1 ou plusieurs fois	`?` 0 ou 1 fois (optionnel)
`{n,m}` Entre n et m fois	`[ ]` Classe de caractères	`( )` Groupe de capture
`|` OU logique	`\d` Chiffre [0-9]	`\w` Mot [a-zA-Z0-9_]
`\s` Espace, tab, newline	`\D` Non-chiffre	`\b` Frontière de mot

Les fonctions du module re

re.match(), re.search(), re.fullmatch()

```python
import re

texte = "Bonjour, je m'appelle Alice et j'ai 25 ans."

# re.match()      cherche UNIQUEMENT en debut de chaine
m = re.match(r"Bonjour", texte)
print(m)            # <re.Match object>
print(m.group())    # Bonjour

m2 = re.match(r"Alice", texte)
print(m2)           # None      Alice n'est pas au debut

# re.search()      cherche n'importe ou dans la chaine
s = re.search(r"Alice", texte)
print(s.group())    # Alice
```

```python
print(s.start())    # 23  (position de debut)
print(s.end())      # 28  (position de fin)

# re.fullmatch()      toute la chaine doit correspondre
code_postal = "75001"
f1 = re.fullmatch(r"\d{5}", code_postal)
print(f1)    # Match

f2 = re.fullmatch(r"\d{5}", "7500X")
print(f2)    # None

# Toujours verifier si le match est None
def valider_code_postal(code):
    if re.fullmatch(r"\d{5}", code):
        return f" '{code}' est un code postal valide"
    return f" '{code}' est invalide"

print(valider_code_postal("75001"))
print(valider_code_postal("7500A"))
print(valider_code_postal("1234"))
```

À retenir

Toujours utiliser des **raw strings** `r"..."` pour les patterns regex. Sans le `r`, Python interprète `\d` comme une séquence d'échappement. Avec `r"\d"`, le backslash est transmis tel quel au moteur regex. C'est une règle absolue pour éviter des bugs subtils et silencieux.

re.findall() et re.finditer()

Trouver toutes les occurrences

```python
import re

texte = "Les numeros sont : 06 12 34 56 78 et 01 23 45 67 89."

# findall()      retourne une LISTE de toutes les
    correspondances
nombres = re.findall(r"\d+", texte)
print(nombres)    # ['06', '12', '34', '56', '78', '01', '23',
    '45', '67', '89']

# Extraire tous les mots en majuscule
phrase = "La POO en Python avec Django et Flask est FUN !"
majuscules = re.findall(r"\b[A-Z]{2,}\b", phrase)
print(majuscules)    # ['POO', 'FUN']

# finditer()      retourne un iterateur de Match objects
```

```python
# Plus efficace que findall() pour de grands textes
log = "ERROR 404 at line 42, WARNING at line 87, ERROR 500 at line 156"
for match in re.finditer(r"ERROR \d+", log):
    print(f"Trouv  : '{match.group()}' a la position {match.start()}")

# Extraire avec des groupes de capture
emails_texte = "Contact : alice@gmail.com ou bob.martin@yahoo.fr"
pattern = r"[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}"
emails = re.findall(pattern, emails_texte)
print(emails)   # ['alice@gmail.com', 'bob.martin@yahoo.fr']

# Extraire des dates
dates_texte = "Ne le 15/03/1990, exam le 25/06/2024"
dates = re.findall(r"\d{2}/\d{2}/\d{4}", dates_texte)
print(dates)   # ['15/03/1990', '25/06/2024']
```

Les groupes de capture

Groupes () — extraire des parties du pattern

```python
import re

# Groupe simple     capturer une partie du match
pattern = r"(\d{2})/(\d{2})/(\d{4})"
texte   = "Date de naissance : 15/03/1990"

m = re.search(pattern, texte)
if m:
    print(m.group(0))   # 15/03/1990  (match complet)
    print(m.group(1))   # 15          (jour)
    print(m.group(2))   # 03          (mois)
    print(m.group(3))   # 1990        (annee)
    print(m.groups())   # ('15', '03', '1990')

# Groupes nommes     (?P<nom>pattern)
pattern_nomme = r"(?P<jour>\d{2})/(?P<mois>\d{2})/(?P<annee>\d{4})"
m2 = re.search(pattern_nomme, texte)
if m2:
    print(m2.group("jour"))    # 15
    print(m2.group("mois"))    # 03
    print(m2.group("annee"))   # 1990
    print(m2.groupdict())
    # {'jour': '15', 'mois': '03', 'annee': '1990'}
```

```python
# Extraire nom et domaine d'un email
email_pattern = r"(?P<user>[\w.+-]+)@(?P<domaine>[\w-]+\.[a-zA-Z]{2,})"
email = "alice.dupont@gmail.com"
m3 = re.search(email_pattern, email)
if m3:
    print(f"Utilisateur : {m3.group('user')}")
    print(f"Domaine     : {m3.group('domaine')}")

# findall() avec groupes     retourne une liste de tuples
log = "2024-03-15 14:32:01 ERROR connexion achouee\n" \
      "2024-03-15 14:33:45 INFO  d marrage r ussi\n"
pattern_log = r"(\d{4}-\d{2}-\d{2}) (\d{2}:\d{2}:\d{2}) (\w+)"
for date, heure, niveau in re.findall(pattern_log, log):
    print(f"[{date} {heure}] Niveau : {niveau}")
```

re.sub() — Remplacer

re.sub() — chercher et remplacer

```python
import re

# Syntaxe : re.sub(pattern, remplacement, texte, count=0)

# Remplacer tous les chiffres par #
texte = "Mon numero est 06 12 34 56 78"
masque = re.sub(r"\d", "#", texte)
print(masque)   # Mon numero est ## ## ## ## ##

# Remplacer avec limite (count)
texte = "chat chat chat chaton"
print(re.sub(r"chat", "chien", texte))         # 4 remplacements
print(re.sub(r"chat", "chien", texte, count=2)) # 2 remplacements

# Normaliser les espaces
brouillon = "Trop   d'espaces    dans   cette   phrase"
propre = re.sub(r"\s+", " ", brouillon).strip()
print(propre)   # Trop d'espaces dans cette phrase

# Utiliser des groupes dans le remplacement avec \1, \2
# Reformater une date de JJ/MM/AAAA vers AAAA-MM-JJ
date = "15/03/2024"
iso = re.sub(r"(\d{2})/(\d{2})/(\d{4})", r"\3-\2-\1", date)
print(iso)   # 2024-03-15

# Avec groupes nommes
```

```python
iso2 = re.sub(
    r"(?P<j>\d{2})/(?P<m>\d{2})/(?P<a>\d{4})",
    r"\g<a>-\g<m>-\g<j>",
    date
)
print(iso2)   # 2024-03-15

# Anonymiser des donnees sensibles
texte_sensible = "Email : alice@gmail.com, tel : 06 12 34 56 78"
anonymise = re.sub(r"[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}",
                   "***@***", texte_sensible)
anonymise = re.sub(r"\d{2}[\s.-]?\d{2}[\s.-]?\d{2}[\s.-]?\d{2}[\s.-]?\d{2}",
                   "** ** ** ** **", anonymise)
print(anonymise)
```

re.split() — Découper

re.split() — diviser avec un pattern

```python
import re

# Diviser sur plusieurs separateurs
texte = "Python;Java,C++|Rust Go"
langages = re.split(r"[;,|\s]+", texte)
print(langages)   # ['Python', 'Java', 'C++', 'Rust', 'Go']

# Diviser une phrase en mots (ignorer ponctuation)
phrase = "Bonjour, monde ! Comment...  a  va ?"
mots = re.split(r"[^\w]+", phrase)
mots = [m for m in mots if m]   # filtrer les vides
print(mots)   # ['Bonjour', 'monde', 'Comment', 'ca', 'va']

# Garder le separateur dans le resultat (groupes)
texte2 = "un1deux2trois3quatre"
parties = re.split(r"(\d)", texte2)
print(parties)   # ['un', '1', 'deux', '2', 'trois', '3', 'quatre']
```

Compiler un pattern — re.compile()

re.compile() — réutiliser un pattern

```python
import re

# Sans compile      pattern recompile a chaque appel
for email in ["alice@gmail.com", "bob@yahoo.fr", "invalid"]:
    if re.match(r"[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}", email):
        print(f"{email}")

# Avec compile      pattern compile UNE SEULE FOIS
# Plus efficace quand on reutilise le meme pattern
EMAIL_RE = re.compile(r"[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}")
TEL_RE   = re.compile(r"0[1-9](?:[\s.-]?\d{2}){4}")
DATE_RE  = re.compile(r"\d{2}/\d{2}/\d{4}")

emails = ["alice@gmail.com", "bob@yahoo.fr",
          "invalide", "charlie@.com"]
for email in emails:
    if EMAIL_RE.fullmatch(email):
        print(f"Email valide    : {email}")
    else:
        print(f"Email invalide  : {email}")

# Les flags      modificateurs de comportement
# re.IGNORECASE (re.I)      insensible a la casse
pattern = re.compile(r"python", re.IGNORECASE)
textes = ["Python", "PYTHON", "python", "PyThOn"]
for t in textes:
    if pattern.search(t):
        print(f"Trouve dans : {t}")

# re.MULTILINE (re.M)      ^ et $ sur chaque ligne
texte_ml = "ligne1\nligne2\nligne3"
debuts = re.findall(r"^\w+", texte_ml, re.MULTILINE)
print(debuts)    # ['ligne1', 'ligne2', 'ligne3']

# re.DOTALL (re.S)      . inclut \n
texte_dot = "debut\nfin"
m = re.search(r"debut.fin", texte_dot, re.DOTALL)
print(m)    # Match trouve
```

Patterns pratiques — Validations courantes

Bibliothèque de patterns utiles

```python
import re

# ==============================================================
# VALIDATIONS COURANTES
# ==============================================================

patterns = {
    "email":         r"^[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}$",
    "tel_fr":        r"^0[1-9](?:[\s.-]?\d{2}){4}$",
    "code_postal":   r"^\d{5}$",
    "url":           r"^https?://[\w.-]+(?:/[\w./?=&%-]*)?$",
    "date_fr":       r"^\d{2}/\d{2}/\d{4}$",
    "heure":         r"^([01]\d|2[0-3]):[0-5]\d$",
    "mot_de_passe": r"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$",
    "ipv4":          r"^(?:\d{1,3}\.){3}\d{1,3}$",
    "hex_couleur":   r"^#[0-9a-fA-F]{6}$",
}

def valider(valeur, type_validation):
    pattern = patterns.get(type_validation)
    if not pattern:
        return False
    return bool(re.match(pattern, valeur))

# Tests
tests = [
    ("alice@gmail.com",    "email"),
    ("pas_un_email",       "email"),
    ("06 12 34 56 78",     "tel_fr"),
    ("75001",              "code_postal"),
    ("https://python.org","url"),
    ("15/03/2024",         "date_fr"),
    ("14:30",              "heure"),
    ("Abc123def",          "mot_de_passe"),
    ("#FF5733",            "hex_couleur"),
]

for valeur, type_v in tests:
    statut = "Bien" if valider(valeur, type_v) else "Faux"
    print(f"{statut} {type_v:15} : {valeur}")
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. Les raw strings `r"..."` ne sont pas optionnelles. J'ai écrit `re.search("\d+", texte)` sans le `r`. Python a interprété `\d` comme une séquence d'échappement inconnue et a levé un `DeprecationWarning`. Avec `r"\d+"`, le backslash est transmis littéralement au moteur regex. Cette subtilité entre l'interprétation Python des chaînes et l'interprétation regex des patterns est fondamentale — et facile à oublier.

2. `re.match()` vs `re.search()` — une différence cruciale. `re.match()` ne cherche qu'au **début** de la chaîne. `re.search()` cherche **partout**. J'ai passé 20 minutes à déboguer un pattern correct qui ne trouvait rien — parce que j'utilisais `match()` au lieu de `search()`. La règle : utiliser `search()` par défaut, et `fullmatch()` pour valider une chaîne entière.

3. Les groupes nommés `(?P<nom>...)` transforment les regex. Passer de `m.group(1)` à `m.group("jour")` change radicalement la lisibilité. Un pattern avec des groupes numérotés est un code mystérieux. Le même pattern avec des groupes nommés devient de la documentation en soi. C'est l'exemple parfait du principe Python : *explicite vaut mieux qu'implicite.*

Résumé du Chapitre 19

Ce que j'ai appris dans ce chapitre :

- Les regex sont un mini-langage pour décrire des **motifs textuels**.
- Toujours utiliser des **raw strings** `r"..."` pour les patterns.
- `re.match()` — début seulement | `re.search()` — n'importe où.
- `re.fullmatch()` — validation de chaîne complète.
- `re.findall()` — liste de toutes les correspondances.
- `re.sub()` — chercher et remplacer avec pattern.
- **Groupes** `()` et **groupes nommés** `(?P<nom>...)` — extraire des parties.
- `re.compile()` — compiler un pattern réutilisé fréquemment.
- Flags : `re.IGNORECASE`, `re.MULTILINE`, `re.DOTALL`.

"Les expressions régulières sont intimidantes au premier regard. Mais une fois maîtrisées, elles transforment des heures de code en une seule ligne."